Foundations — how this project works, explained from zero

Written for someone in a first electronics or embedded course. No prior knowledge of SPI, LED drivers, web servers or build systems is assumed. Every term is explained the first time it appears.

If you already know what a shift register is, skip to Part 4.

Part 1 — What we are actually building

A small sign that displays Arabic text on a grid of red LEDs. You type the text on your phone, and the letters appear on the sign.

Three pieces of hardware:

```
┌─────────┐      ┌──────────┐       ┌────────────┐
│  PHONE  │ WiFi │  ESP32   │ wires │ LED MATRIX │
│(browser)│═════▶│(computer)│══════▶│   (8 x 8)  │
└─────────┘      └──────────┘       └────────────┘
```

- **The phone** is where you type. Nothing is installed on it — it just opens a web page.
- **The ESP32** is a small computer with WiFi built in. It costs a few dollars.
- **The LED matrix** is 64 red LEDs arranged in an 8×8 grid, with a driver chip behind it.

The rest of this document explains each piece, and how they talk to each other.

Part 2 — The connection diagram

Five wires connect the ESP32 to the LED module. That's all.

```
     ESP32                          MAX7219 LED MODULE
┌─────────────┐                   ┌──────────────────┐
│             │                   │ ● ● ● ● ● ● ● ●  │
│  VIN   ─────┼───── red ────────▶│ VCC   ● ● ● ● ●  │
│  GND   ─────┼───── black ──────▶│ GND   (64 LEDs)  │
│  GPIO13 ────┼───── green ──────▶│ DIN   ● ● ● ● ●  │
│  GPIO14 ────┼───── blue ───────▶│ CLK   ● ● ● ● ●  │
│  GPIO27 ────┼───── yellow ─────▶│ CS    ● ● ● ● ●  │
│             │                   │                  │
└─────────────┘                   └──────────────────┘
```

What each wire does

Wire	Full name	Job
VCC	Voltage Common Collector	Power in — supplies 5 volts to run the LEDs
GND	Ground	The return path. Every circuit needs one
DIN	Data In	Carries the actual information, one bit at a time
CLK	Clock	A metronome. Says "*now*" — read the data bit
CS	Chip Select	Says "*I'm talking to you*" and "*message finished*"

Why a clock wire at all? The data wire only carries a voltage — high or low. If you send ten `1` bits in a row, the wire just sits high the whole time. The receiver has no way to know whether that's three bits or ten. The clock solves it: every time the clock ticks, the receiver samples the data wire once. Ten ticks, ten bits. No ambiguity.

Think of it as spelling a word aloud over a bad phone line. The letters are the data; saying "next… next… next…" is the clock.

GND is not optional. Voltage is always a *difference* between two points. Saying "the data wire is at 3.3 V" is meaningless unless both chips agree on what 0 V is. The ground wire is that agreement. Without it, the data wire's voltage means nothing to the receiver, and the display behaves randomly rather than failing cleanly — which makes it a confusing fault to diagnose.

Why 5 V and not 3.3 V

The ESP32 runs internally at 3.3 V, but the LED module wants 5 V.

LEDs need a certain voltage before they conduct at all. Red LEDs need roughly 1.8–2 V, and the driver chip needs some headroom above that. At 3.3 V the LEDs are dim, and — more subtly — the driver's idea of "this input counts as a logic 1" may no longer match what the ESP32 outputs. You get flicker and garbage rather than an honest failure.

The ESP32 board has a **VIN** pin that passes through the 5 V coming from USB, which is exactly what the module wants.

One pin choice that matters

CS is on **GPIO27** even though **GPIO12** sits physically closer to the other wires.

GPIO12 is a **strapping pin**. When the ESP32 powers up, it reads the voltage on a few specific pins to decide how to configure itself — GPIO12 selects the internal flash memory voltage. It must read LOW at that instant. If a wire holds it high at power-on, the chip picks the wrong flash voltage and **refuses to boot**. The board looks dead, and nothing about the symptom points at the wiring.

Lesson worth internalising early: on microcontrollers, a few pins have special jobs during boot. Check the datasheet before choosing pins, not after.

Part 3 — How the LED matrix works

The wiring problem

An 8×8 matrix has 64 LEDs. Controlling each one individually would need 64 wires plus ground. The module has 16 pins.

The trick is in the wiring:

```
        COLUMN 0   COLUMN 1   COLUMN 2  ...
           │          │          │
ROW 0 ─────┼─▶|───────┼─▶|───────┼─▶|──
           │          │          │
ROW 1 ─────┼─▶|───────┼─▶|───────┼─▶|──
           │          │          │
ROW 2 ─────┼─▶|───────┼─▶|───────┼─▶|──
           │          │          │
```

Every LED sits at the crossing of one row wire and one column wire. 8 rows + 8 columns = **16 wires control 64 LEDs**.

An LED lights only when current flows through it — which needs its row driven high *and* its column pulled low. So to light the LED at row 2, column 5: set row 2 high, pull column 5 low.

The catch, and the trick

You cannot display an arbitrary picture this way all at once. If you set rows 0 and 1 high, and pull columns 3 and 6 low, you light **four** LEDs — (0,3), (0,6), (1,3), (1,6) — not the two you wanted. The rows and columns can't be aimed independently.

The solution is **multiplexing**: never light more than one row at a time.

```
time →


┌───────┬─────────┬─────────┬─────────┐
│ row 0 │ row 1   │ row 2   │ row 3   │  ... and repeat
│ columns │ columns │ columns │ columns │
│ for r0  │ for r1  │ for r2  │ for r3  │
└───────┴─────────┴─────────┴─────────┘
   1 ms      1 ms      1 ms      1 ms
```

Light row 0 with its correct column pattern. Turn it off. Light row 1 with *its* pattern. And so on through all eight, then start again.

Each row is lit only ⅛ of the time — but if the whole cycle repeats faster than about 50 times per second, your eye cannot follow it. **Persistence of vision** blends the flashes into a steady image.

This is the same effect that makes film look like motion rather than a sequence of stills. Every LED display, phone screen and television relies on it.

Cost of the trick: each LED is off ⅞ of the time, so the display is dimmer than if all LEDs ran continuously. Drivers compensate by pushing more current during each brief on-period.

Part 4 — What a register is

This is the concept everything else rests on.

A **register** is a small piece of memory *inside a chip*. Not the microcontroller's RAM — memory built into the peripheral chip itself, usually only a few bits wide.

The mental model: a register is **a row of switches that stay where you put them**.

```
  bit 7   6   5   4   3   2   1   0
 ┌───┬───┬───┬───┬───┬───┬───┬───┐
 │ 1 │ 0 │ 1 │ 1 │ 0 │ 0 │ 1 │ 0 │   an 8-bit register
 └───┴───┴───┴───┴───┴───┴───┴───┘
```

Write `10110010` into it and it holds `10110010` until you write something else — even if you walk away. In an LED driver, each bit typically controls one LED: 1 = on, 0 = off.

Why chips use registers. A chip needs to be told what to do, but adding a pin for every setting would make it enormous. Instead it exposes a few registers and one small serial connection. Configuration becomes "write this number to that register" rather than "add another wire".

A chip usually has several registers, each with a **number** (its address) and a **job**:

Address	Job
1	The 8 LEDs of row 0
2	The 8 LEDs of row 1
…	…
10	Brightness
12	On/off

So "display something" means: write the right number into the right register.

Part 5 — The famous 8-bit register: 74HC595

If you have met a shift register in a course, it was almost certainly this one. Understanding it makes the MAX7219 obvious.

What a shift register does

A **shift register** takes bits in **one at a time** (serial) and makes them available **all at once** (parallel).

```
DATA ──▶ ┌───┬───┬───┬───┬───┬───┬───┬───┐
         │ b7│ b6│ b5│ b4│ b3│ b2│ b1│ b0│
         └─┬─┴─┬─┴─┬─┴─┬─┴─┬─┴─┬─┴─┬─┴─┬─┘
           ▼   ▼   ▼   ▼   ▼   ▼   ▼   ▼
          Q7  Q6  Q5  Q4  Q3  Q2  Q1  Q0     8 output pins
```

Each clock tick pushes the incoming bit in at one end, and everything already inside shifts one place along — like people shuffling down a bench as someone new sits at the end. After 8 ticks, your 8 bits are lined up, and a **latch** pulse copies them to the output pins all at once.

Why the latch matters: without it, the outputs would flicker through garbage during the shifting. The latch means the outputs only change when you say the data is complete.

Why this is useful: three microcontroller pins (data, clock, latch) become eight outputs. Chain two chips and three pins become sixteen — the shifted-out bits from the first simply feed the second.

Driving LEDs with a 74HC595 — and what you must add

The 74HC595 is a *general-purpose* chip. It knows nothing about LEDs. To build an 8×8 display with it you must supply everything else yourself:

You must add	Why
A resistor per LED (8+)	The 595 doesn't limit current; without resistors the LEDs burn out
A second chip for rows	One 595 drives 8 columns; rows need their own driver
Transistors	A whole row of LEDs draws more current than the chip can source
Multiplexing code	*Your* program must cycle the rows fast enough, forever
Brightness logic	Any dimming you want, you write yourself

That multiplexing loop is the real burden. It must run continuously — a few milliseconds of delay elsewhere in your program shows up as visible flicker. Your code is now permanently responsible for refreshing the display, on top of whatever else it's meant to do.

Part 6 — MAX7219: the same idea, specialised

The MAX7219 is a shift register **plus everything you would otherwise have had to add**.

```
74HC595                        MAX7219
───────                        ───────

shift register           ────▶ shift register
(you add resistors)      ────▶ constant-current drivers built in
(you add row driver)     ────▶ row driver built in
(you write multiplexing) ────▶ multiplexing in hardware
(you write dimming)      ────▶ 16 brightness levels built in
(you track state)        ────▶ 8 registers hold the image
```

What it does for you

Constant current, one resistor total. A single resistor sets the current for all 64 LEDs. Not 64 resistors — one. Every LED gets the same brightness regardless of how many are lit, which the 595 cannot do.

Multiplexing in silicon. The chip cycles the rows itself, continuously, forever. Your program is free the instant the data is sent. **This is the big one** — the whole refresh burden disappears.

Memory for the image. Eight registers hold the current picture. Write once; the chip keeps displaying it. Nothing to maintain.

Brightness control. Write a number 0–15 to the brightness register. No extra circuitry.

The comparison

	74HC595	MAX7219
Purpose	General shift register	LED matrix driver
Resistors needed	One per LED	One, total
Multiplexing	Your code, forever	Hardware
Brightness	Your circuit	Built in, 16 levels
Remembers image	No	Yes
Chips for one 8×8	2+ plus transistors	1
Cost	Cheaper per chip	Cheaper overall for this job
Flexibility	Drives anything	LEDs only

Which should you use? The 595 when you need generic outputs — relays, other chips, mixed loads. The MAX7219 when you are driving LED matrices, because it removes an entire category of work.

The 595 is the better *teaching* chip precisely because it makes you do the work by hand. The MAX7219 is the better *building* chip because you don't have to.

Part 7 — The protocol: SPI

Both chips are loaded the same way, using **SPI** — Serial Peripheral Interface.

"Serial" means bits travel **one after another on a single wire**, rather than eight wires side by side.

The three signals

```
ESP32 (master)                        MAX7219 (slave)
    CLK  ────┐ ┌┐ ┌┐ ┌┐ ┌────────▶ CLK
             └─┘└─┘└─┘└─┘

    DIN  ────  1   0   1   1  ─────▶ DIN

    CS   ──╲__________________╱──   CS
            └── message ──┘
```

- **CLK** — the master generates ticks. On each tick the slave samples the data line.
- **DIN** — one bit per tick.
- **CS** — held low for the whole message. Its **rising edge** at the end means "*message complete, act on it.*"

CS is the latch from Part 5, under a different name. The MAX7219 collects bits while CS is low, and applies them when CS goes high.

Master and slave: the master generates the clock and starts every exchange. Slaves only respond. Here the ESP32 is master and the MAX7219 is slave — which is why the MAX7219 can never volunteer information. It has **no output wire at all.**

That fact has a real consequence for this project: you cannot ask the module how it is wired internally. There is no path for an answer. Anything you need to know about it must be found by trying something and *looking at the panel*.

What the MAX7219 expects

Sixteen bits per message:

```
┌──────────┬──────────┬────────────────┐
│  4 bits  │  4 bits  │     8 bits     │
│  ignored │ register │      data      │
│          │ address  │                │
└──────────┴──────────┴────────────────┘
```

So sending `0x01, 0xFF` means "*register 1 (row 0), all eight bits on*" — the top row lights fully.

The 74HC595 has no address field at all; you just push 8 bits and latch. The MAX7219 spends 4 extra bits on an address so one chip can hold several independent registers. That is the difference between a generic part and a specialised one, visible right in the wire format.

Chaining

Both chips chain the same way: bits shifted out of chip 1 flow into chip 2.

A consequence that surprises everyone: the word you send **first** ends up in the **last** chip in the chain. Everything you send has to travel through the earlier chips to reach the later ones, so it pushes previous data along ahead of it. When writing code for a chain, you send data in reverse order.

Part 8 — How can an ESP32 serve a web page?

This is the part that seems magical and isn't.

What a web server actually is

A web server is just **a program that waits for a request and sends text back**. That's the whole job. Nothing about it requires a big computer.

When you open `http://192.168.4.1/`, your browser:

1. Opens a connection to that address
2. Sends a short text message: `GET / HTTP/1.1`
3. Waits
4. Receives text back — HTML — and draws it

The server's side is: listen, receive `GET /`, send back the HTML. A few hundred lines of code, and the ESP32 has plenty of room for it.

The layers involved

```
┌────────────────────────────────────┐
│  Your HTML page (stored in flash)  │  ← what you wrote
├────────────────────────────────────┤
│  HTTP server   — understands GET/POST │  ← library
├────────────────────────────────────┤
│  TCP/IP stack  — addresses, packets   │  ← library (lwIP)
├────────────────────────────────────┤
│  WiFi driver   — radio, 802.11        │  ← chip + library
├────────────────────────────────────┤
│  WiFi radio hardware                  │  ← silicon
└────────────────────────────────────┘
```

You write the top layer. Everything beneath ships with the ESP-IDF. This is what "batteries included" means for an SDK.

Access Point mode — no router needed

WiFi chips work in two modes:

- **Station (STA)** — joins an existing network. What your phone normally does.
- **Access Point (AP)** — *becomes* a network that others join. What a router does.

The ESP32 can do either. Here it runs as an **access point**, so it creates its own WiFi network that your phone joins directly. **No router, no internet, no SIM.** The sign works standing alone in a shop.

How the phone gets an address

When your phone joins, it needs an IP address. The ESP32 runs a small **DHCP server** — the same service your home router runs — that hands one out, typically `192.168.4.2`, and tells the phone that the sign itself is at `192.168.4.1`.

DHCP also tells the phone useful extras, including which server to use for looking up names. That detail matters more than it sounds: if that information is missing, the phone can join successfully and still behave as though the network is broken.

Where the page is stored

The HTML file is compiled **into the firmware** and lives in the ESP32's flash memory alongside the program. There is no SD card and no filesystem to manage. When a request arrives, the server sends those bytes straight out of flash.

Part 9 — Firmware toolchains: Arduino vs PlatformIO vs ESP-IDF

These three are constantly confused, partly because they are not the same *kind* of thing.

The clarification that resolves most confusion

- **Arduino** is a **framework** — a set of simplified functions (`digitalWrite`, `setup`, `loop`).
- **ESP-IDF** is a **framework** — Espressif's official SDK for the ESP32.
- **PlatformIO** is a **build tool** — it is *not* a competing framework. It compiles and uploads projects, and it can build **either** Arduino **or** ESP-IDF projects.

So "Arduino vs ESP-IDF" is a real choice. "PlatformIO vs ESP-IDF" is a category error — PlatformIO can build ESP-IDF projects.

Arduino

```
void setup() { pinMode(13, OUTPUT); }
void loop()  { digitalWrite(13, HIGH); delay(500); }
```

Good: the easiest possible start, huge library ecosystem, works across many boards.

Limits: it hides the hardware, which becomes a problem when you need something it didn't anticipate. Written for 8-bit chips originally, so features specific to the ESP32 — dual cores, the task scheduler, deep configuration — are awkward or absent.

ESP-IDF

Espressif's own SDK — the one Arduino-for-ESP32 is itself built on top of.

```
void app_main(void) {
    gpio_set_direction(GPIO_NUM_13, GPIO_MODE_OUTPUT);
    gpio_set_level(GPIO_NUM_13, 1);
}
```

Good: full access to every feature. Includes **FreeRTOS**, a real-time operating system, so you can run several tasks concurrently — one scrolling the display while another serves web pages, with no manual interleaving. Configuration through `menuconfig` rather than editing code.

Limits: more to learn, more setup, more concepts before the first LED blinks.

PlatformIO

An extension for VS Code that manages toolchains, libraries and builds. You choose the framework in a config file:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf      ; or "arduino"
```

Good: pleasant editor integration, tidy dependency handling.

Limits: an extra layer between you and the official tools, which can lag behind or obscure errors.

Why this project uses ESP-IDF

Requirement	Why ESP-IDF
WiFi access point + DHCP + HTTP server	All included and properly integrated
Scroll the display *while* serving pages	FreeRTOS tasks make this straightforward
Configurable pins and panel size without editing code	`menuconfig`
Precise control of SPI and timing	Direct access to the drivers

With Arduino, most of this is possible but fought for. With ESP-IDF it is what the SDK is *for*.

Part 10 — Installing ESP-IDF

```
# 1. Dependencies (Ubuntu/Debian)
sudo apt install git wget flex bison gperf python3 python3-pip \
                 python3-venv cmake ninja-build ccache libffi-dev \
                 libssl-dev dfu-util libusb-1.0-0

# 2. Download the SDK
mkdir -p ~/esp && cd ~/esp
git clone -b v5.3.2 --recursive https://github.com/espressif/esp-idf.git

# 3. Install the compiler for the ESP32
cd ~/esp/esp-idf
./install.sh esp32

# 4. Activate it — needed in EVERY new terminal
. ./export.sh
```

Why `--recursive`? ESP-IDF includes other projects (FreeRTOS, lwIP, mbedTLS) as *submodules*. Without that flag you download an incomplete SDK, and the build fails confusingly.

Why `export.sh` every time? It adds the compiler and `idf.py` to your `PATH` for that terminal only. It deliberately doesn't install globally, so you can keep several IDF versions side by side. Forgetting it produces `idf.py: command not found` — that's all it means.

Serial port permission

```
ls -l /dev/ttyUSB0           # does the board appear?
groups | grep dialout        # are you allowed to use it?
sudo usermod -aG dialout $USER   # if not — then log out and back in
```

On Linux, serial ports belong to the `dialout` group. Without membership you get "permission denied", which looks like a broken board and isn't.

You never press a button to flash. The USB-serial chip on the board uses two control lines to reset the ESP32 and put it into its bootloader automatically. The flashing tool handles it.

Part 11 — Using ESP-IDF

```
. ~/esp/esp-idf/export.sh      # once per terminal
cd firmware

idf.py set-target esp32        # once per project
idf.py menuconfig              # change settings — arrow keys, Enter, S to save, Q to quit
idf.py build                   # compile
idf.py -p /dev/ttyUSB0 flash   # upload
idf.py -p /dev/ttyUSB0 monitor # watch the board's messages (Ctrl-] to exit)

idf.py -p /dev/ttyUSB0 flash monitor   # the everyday command
```

What menuconfig is

A settings menu compiled into your build. Instead of editing constants in source files, options are declared in a file called `Kconfig`, and `menuconfig` presents them as a menu.

This project declares things like which GPIO pins the display uses, how many modules are connected, and the WiFi network name. Changing the wiring means changing a menu entry, not hunting through code.

Files you will meet

File	Purpose
`CMakeLists.txt`	Lists source files — the build recipe
`Kconfig.projbuild`	Declares your `menuconfig` options
`sdkconfig.defaults`	Starting values for settings, safe to commit
`sdkconfig`	The **generated** current settings — do not commit
`build/`	Compiler output — do not commit

Part 12 — How the code is organised

Not what every line does — how the pieces fit. This structure is common to almost all C projects.

Three kinds of thing

1. Includes — bringing in code someone else wrote

```
#include <string.h>         // standard C library
#include "driver/spi_master.h"  // ESP-IDF's SPI driver
#include "max7219.h"        // our own module
```

`#include` means "*paste that file's declarations here*". Angle brackets mean a system library; quotes mean a file in this project.

2. Functions — named blocks of work

```
void fb_set_pixel(framebuffer_t *fb, int x, int y, bool on)
{
    ...
}
```

Read it as: *returns nothing* (`void`), *called* `fb_set_pixel`, *takes* a framebuffer, an x, a y, and on/off. Called from elsewhere as `fb_set_pixel(&buffer, 3, 5, true);`

3. Modules — files grouped by responsibility

A module is a **pair** of files:

- `max7219.h` — the **header**: what this module offers. A menu.
- `max7219.c` — the **implementation**: how it actually does it. The kitchen.

Other files `#include` the header and call the functions. They never need to read the `.c`.

Why split them? So you can change *how* something works without anything else caring, as long as the menu stays the same. This is the single most useful idea in software structure, and it is what makes a project survivable as it grows.

The modules in this project

Module	Responsibility
`main.c`	Startup and coordination
`max7219`	Talking to the LED driver chip
`framebuffer`	Storing the picture in memory
`frame_rx`	Receiving pictures over the cable
`wifi_ap`	Creating the WiFi network
`http_ui`	Serving the web page
`font5x7` / `text5x7`	Drawing Latin letters, for testing
`www/index.html`	The page your phone opens

Each does **one** job. `max7219.c` knows about the chip but nothing about WiFi. `wifi_ap.c` knows about networks but nothing about LEDs. That separation is deliberate — it means you can understand one file without holding the whole project in your head.

Where the program starts

Ordinary C programs start at `main()`. ESP-IDF programs start at **`app_main()`**, because the operating system runs first and then calls your code.

```
void app_main(void)
{
    max7219_init(...);      // 1. wake up the display
    fb_init(...);           // 2. make room for a picture
    wifi_ap_start(...);     // 3. create the WiFi network
    http_ui_start(...);     // 4. start serving the page
    // ... then the program keeps running, handling events
}
```

Notice `app_main` doesn't loop forever. Under **FreeRTOS**, work happens in independent **tasks** — one refreshing the display, one handling web requests — that the operating system switches between. `app_main` sets them up and finishes.

That is precisely what would be painful in Arduino's single `loop()`, and it is the main reason this project uses ESP-IDF.

Part 13 — Putting the whole chain together

```
 1. You type Arabic on your phone
             ↓
 2. The web page turns the letters into a grid of on/off dots
             ↓
 3. The dots are packed into bytes — 8 dots per byte
             ↓
 4. Sent over WiFi to the ESP32
             ↓
 5. The ESP32 stores them in its framebuffer
             ↓
 6. It sends them over SPI to the MAX7219 registers
             ↓
 7. The MAX7219 multiplexes the rows continuously
             ↓
 8. Your eye blends the flashes into steady letters
```

Every step in that chain has now been explained: what a register is (4), how the matrix multiplexes (3), which protocol carries the bits (7), why the ESP32 can host a page (8), and how the code is arranged to do it (12).

Glossary

Term	Meaning
Access Point (AP)	A device that creates a WiFi network for others to join
Bit / Byte	A single 0-or-1; eight bits together
Chip Select (CS)	Signal meaning "I'm talking to you" / "message finished"
DHCP	Service that hands out IP addresses to devices joining a network
Firmware	Software stored inside a device rather than on a disk
Flash memory	Storage that survives power-off; holds the program
FreeRTOS	Small operating system letting several tasks run concurrently
GPIO	General-Purpose Input/Output — a pin your program controls
Latch	Copies collected bits to the outputs all at once
Multiplexing	Sharing wires by using them for different things in turn
Persistence of vision	The eye blending fast flashes into a steady image
Register	Small memory inside a chip that holds a setting or data
Serial	Sending bits one after another on one wire
Shift register	Chip that takes bits in one at a time, outputs them together
SPI	Serial protocol using clock, data and select lines
Strapping pin	A pin read at power-on to configure the chip
Toolchain	The compiler and tools that turn source code into firmware